

From a number to a decision

The same loop as Lab 7, with one new line — a sigmoid — and a different loss. Train a logistic neuron from scratch on a 2-D dataset and draw the boundary it learned.

Scaffolded lab

20 minutes

Pure NumPy

Plot the boundary

Before you start

Setup · given

1 · the sigmoid

2 · binary cross-entropy

3 · train the neuron

4 · draw the boundary

5 · why not squared error?

Reflection

If you finish early

Before you start

Open the provided `lab08.ipynb`. The data, the plotting and the checks are given; the sigmoid, the loss and the gradient step are yours.



By the end of this lab you can

- Write the sigmoid and binary cross-entropy in NumPy.
- Train a logistic neuron by gradient descent on two features.
- Draw the learned decision boundary from the weights.
- Explain why squared error is the wrong loss for a classifier.

LAB 8

Setup · given

Forty points in two classes, generated with a fixed seed so everyone has the same data.

given

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(11)
X = np.vstack([rng.normal((-1.0, -0.6), 0.7, (20, 2)),
               rng.normal(( 1.0,  0.8), 0.7, (20, 2))])
y = np.array([0] * 20 + [1] * 20, dtype=float)

plt.scatter(X[:, 0], X[:, 1], c=y, cmap="coolwarm", edgecolor="k")
plt.xlabel("x1"); plt.ylabel("x2"); plt.title("Two classes")
plt.show()
```

TASK 1 · 3 MINUTES

the sigmoid

1

One line. It squashes any number into the interval from 0 to 1, so the output can be read as a probability.

your turn

PYTHON

```
def sigmoid(z):
    return None    # TODO

print(sigmoid(0.0), sigmoid(np.array([-5.0, 5.0])))
assert sigmoid(0.0) == 0.5
assert np.allclose(sigmoid(np.array([-5.0, 5.0])), [0.00669, 0.99331], atol=1e-5)
```

what it should print

OUTPUT

```
0.5 [0.00669285 0.99330715]
```

TASK 2 · 3 MINUTES

binary cross-entropy

2

$$L = -\mathbb{E} \left[y \log p + (1-y) \log(1-p) \right]$$

Add a tiny `eps` inside both logarithms so that a probability of exactly 0 or 1 does not produce `-inf`.

your turn

PYTHON

```
def bce(p, y, eps=1e-12):  
    return None      # TODO  
  
print(round(bce(np.full(40, 0.5), y), 4))      # a neuron that knows nothing  
assert np.isclose(bce(np.full(40, 0.5), y), np.log(2))
```

what it should print

OUTPUT

0.6931



🔑 **0.6931 is a number to remember.** It is $\log 2$, the loss of a model that says 50–50 about everything. A training run whose loss sits at 0.693 has learned nothing, and you will see exactly that on Friday's XOR demonstration.

TASK 3 · 7 MINUTES · THE MAIN TASK

train the neuron

3

For a sigmoid with cross-entropy, the chain rule collapses to something very short: the gradient of the loss with respect to the score `z` is just `p - y`. From there, `dw` and `db` follow the same pattern as Lab 7.

your turn

PYTHON

```
w = np.zeros(2)
b = 0.0
lr = 0.5
history = []

for epoch in range(500):
    z = X @ w + b
    p = sigmoid(z)
    history.append(bce(p, y))
    dz = None      # TODO
    dw = None      # TODO: shape (2,)
    db = None      # TODO
    w = w - lr * dw
    b = b - lr * db

accuracy = np.mean((sigmoid(X @ w + b) > 0.5) == y)
print("w =", np.round(w, 3), " b =", round(b, 3), " loss =", round(history[-1], 4), "
accuracy =", accuracy)
assert history[-1] < history[0] and accuracy ≥ 0.9
```

one version that works

PYTHON

```
w = np.zeros(2); b = 0.0; lr = 0.5; history = []
for epoch in range(500):
    z = X @ w + b
    p = sigmoid(z)
    history.append(bce(p, y))
    dz = p - y
    dw = X.T @ dz / len(X)
    db = np.mean(dz)
    w = w - lr * dw
    b = b - lr * db
accuracy = np.mean((sigmoid(X @ w + b) > 0.5) == y)
print("w =", np.round(w, 3), " b =", round(b, 3), " loss =", round(history[-1], 4), "
accuracy =", accuracy)
assert history[-1] < history[0] and accuracy ≥ 0.9
```

what it should print

OUTPUT

```
w = [3.979 4.171]  b = -0.408  loss = 0.0119  accuracy = 1.0
```

TASK 4 · 4 MINUTES

draw the boundary

4

The neuron says 0.5 exactly where $w_1x_1 + w_2x_2 + b = 0$. Solve that for x_2 — one line of algebra, one line of code.

your turn

PYTHON

```
x1 = np.linspace(-3, 3, 50)
x2 = None      # TODO: the boundary, solved for x2

plt.scatter(X[:, 0], X[:, 1], c=y, cmap="coolwarm", edgecolor="k")
plt.plot(x1, x2, "k--", label="p = 0.5")
plt.xlabel("x1"); plt.ylabel("x2"); plt.ylim(-3, 3); plt.legend(); plt.title("The learned boundary")
plt.show()
```

one version that works

PYTHON

```
x1 = np.linspace(-3, 3, 50)
x2 = -(w[0] * x1 + b) / w[1]
plt.plot(x1, x2)
```

TASK 5 · 3 MINUTES

why not squared error?

5

Run this, given. It asks one question: when the neuron is **confidently wrong** (says 0.001 for a point whose label is 1), how hard does each loss push?

given

PYTHON

```
p, target = 0.001, 1.0
push_bce = p - target          # cross-entropy, through the sigmoid
push_mse = 2 * (p - target) * p * (1 - p)  # squared error, through the sigmoid
print("cross-entropy push:", round(push_bce, 4))
print("squared-error push:", round(push_mse, 6))
```

what it should print

OUTPUT

```
cross-entropy push: -0.999
squared-error push: -0.001996
```

Write one sentence: which loss would leave a badly wrong neuron almost stuck, and which factor in the second line is to blame?

Your answer:

Reflection



Write it in the notebook, in your own words

Put this loop next to Lab 7's. What stayed **structurally the same**, and which two things changed?

Your answer:

If you finish early

A. Plot `history` on a log axis. Does the loss ever reach zero on this data? Why can it not, when two of the points overlap the other class?

B. Move one blue point deep into the red cloud and retrain. How far does the boundary move for one outlier?

your turn

PYTHON

```
# TODO (optional extensions A, B)
```

